

Algorithm Visualizer Pro

Comprehensive Function Library & Feature Reference Guide

This document serves as the official reference manual for the internal features, algorithms, custom Domain Specific Language (DSL) APIs, and internal JavaScript functions utilized within the Algorithm Visualizer Pro web application.

1. Core Sorting Algorithms Library

The following functions represent the core sorting logic executed within the application's Code Library and Visualizer engine.

Bubble Sort

$O(n^2)$ Time | $O(1)$ Space

```
function bubbleSort(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length - 1; i++) {
    for (let j = 0; j < a.length - i - 1; j++) {
      if (a[j] > a[j + 1]) {
        [a[j], a[j + 1]] = [a[j + 1], a[j]];
      }
    }
  }
  return a;
}
```

Selection Sort

$O(n^2)$ Time | $O(1)$ Space

```
function selectionSort(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length - 1; i++) {
    let minIdx = i;
    for (let j = i + 1; j < a.length; j++) {
      if (a[j] < a[minIdx]) minIdx = j;
    }
    if (minIdx !== i) [a[minIdx], a[i]] = [a[i], a[minIdx]];
  }
  return a;
}
```

Merge Sort

$O(n \log n)$ Time | $O(n)$ Space

```
function mergeSort(arr) {
  if (arr.length <= 1) return arr;
  const mid = Math.floor(arr.length / 2);
  const left = mergeSort(arr.slice(0, mid));
  const right = mergeSort(arr.slice(mid));
  return merge(left, right);
}

function merge(left, right) {
  const result = [];
  let i = 0, j = 0;
  while (i < left.length && j < right.length) {
    if (left[i] <= right[j]) result.push(left[i++]);
    else result.push(right[j++]);
  }
  return result.concat(left.slice(i)).concat(right.slice(j));
}
```

Quick Sort

$O(n \log n)$ Time | $O(\log n)$ Space

```
function quickSort(arr) {
  const a = [...arr];
  function partition(low, high) {
    const pivot = a[high];
    let i = low - 1;
    for (let j = low; j < high; j++) {
      if (a[j] < pivot) {
        i++;
        [a[i], a[j]] = [a[j], a[i]];
      }
    }
    [a[i + 1], a[high]] = [a[high], a[i + 1]];
    return i + 1;
  }
  function sort(low, high) {
    if (low < high) {
      const pi = partition(low, high);
      sort(low, pi - 1);
      sort(pi + 1, high);
    }
  }
  sort(0, a.length - 1);
  return a;
}
```

2. Pathfinding Algorithms Library

These functions power the pathfinding arena, operating on grid coordinates and navigating around obstacles.

Breadth-First Search (BFS)

Guarantees Shortest Path (Unweighted)

```
function bfs(grid, start, end) {
  const queue = [start];
  const visited = new Set([`${start[0]}`, `${start[1]}`]);
  const parent = {};
  // Implementation dequeues nodes, checks neighbors,
  // and traces path back using parent map upon finding target.
}
```

Dijkstra's Algorithm / A* Search

Guarantees Shortest Path (Weighted)

```
function aStar(grid, start, end) {  
  // Uses Manhattan distance heuristic  
  const h = (r, c) => Math.abs(r - end[0]) + Math.abs(c - end[1]);  
  let open = [[h(...start), ...start]]; // [f-score, r, c]  
  const dist = { [`${start[0]}`, `${start[1]}`]: 0 };  
  // Explores nodes ordered by lowest f-score (cost + heuristic)  
}
```

3. Custom Algorithm DSL Reference

The Domain Specific Language (DSL) allows users to write custom algorithms in JavaScript. The compiler intercepts specific function calls to generate visualization frames instantly.

n

A globally injected integer representing the length of the currently loaded array.

compare(i, j)

Parameters: *i* (index), *j* (index)

Returns: Number (> 0 if $\text{arr}[i] > \text{arr}[j]$, < 0 if $\text{arr}[i] < \text{arr}[j]$)

Action: Highlights bars at indices *i* and *j* in red and increments the global comparison counter.

swap(i, j)

Parameters: *i* (index), *j* (index)

Action: Physically swaps the elements at the specified indices in the array, triggers a blue highlight animation, and increments the swap counter.

get(i)

Parameters: *i* (index)

Returns: The numerical value at array index *i*.

set(i, value)

Parameters: *i* (index), *value* (number)

Action: Overwrites the value at index *i* with *value*, triggering an active bar animation. Used for algorithms like Merge Sort or Insertion Sort where direct swaps aren't used.

4. Core Engine Controllers

These primary functions orchestrate the web application's states, animations, and toolsets.

startSort()

Initiates the main visualization sequence by fetching pre-computed frames from the selected algorithm.

runCompare()

Triggers the dual-arena comparison mode, executing two algorithms simultaneously with scaled timings.

initDeepDive()

Compiles Deep Dive frames with contextual insights, local variable tracking, and line-by-line code mapping.

runPathfind()

Executes the pathfinding logic on the active grid, pushing visited nodes and frontier states to the animation queue.

dslCompileAndRun(arr)

Secures user code within a strict context block, injects the DSL APIs, caps execution frames at 20,000 to prevent infinite loops, and renders the result.

5. Internal JavaScript API Directory

The complete catalog of all utility, interface, and logic functions utilized in the Algorithm Visualizer.

Global State & Navigation

toggleTheme()

Switches the application between Light and Dark mode.

switchTab(tabId)

Handles routing between Visualize, Compare, Deep Dive, Pathfinding, Custom DSL, and Library tabs.

togglePause()

Pauses or resumes any currently active sorting or animation sequence.

stepForward()

Advances the animation by a single frame while paused.

checkPause()

Async utility hook to await unpausing within animation loops.

resetControls()

Resets all global interaction buttons to their default active states.

UI & Visualization

renderAlgoInfoCard(algoKey)

Renders complexity tables and metadata for the selected algorithm.

heapClear() / heapRenderBuffer() / heapSetNote()

Manages the auxiliary memory visualizer used in Merge/Radix/Counting sorts.

resetChart() / pushChartPoint() / drawChart()

Controls the HTML5 Canvas rendering for the Live vs Theoretical complexity chart.

celebrate(algoName)

Triggers the confetti and success banner upon sequence completion.

init() / generateRandom() / setCustomArray()

Bootstraps the main visualizing area with fresh array datasets.

renderVisuals(arr, barContainerId, boxContainerId)

Generates the DOM elements for array bars and number boxes.

updateCodeView() / updateProgress()

Syncs the side panels with current animation steps.

Frame Generators

getBubbleSortFrames(arr)

Yields swap/compare frames for Bubble Sort.

getSelectionSortFrames(arr)

Yields frames for Selection Sort.

getInsertionSortFrames(arr)

Yields frames for Insertion Sort.

getMergeSortFrames(arr)

Yields overwrite/compare frames for Merge Sort.

getQuickSortFrames(arr)

Yields partition/swap frames for Quick Sort.

getHeapSortFrames(arr)

Yields frames for Heap Sort building and extraction.

getShellSortFrames(arr)

Yields gap-based insertion frames for Shell Sort.

getCountingSortFrames(arr) / getRadixSortFrames(arr)

Yields frequency accumulation and rewrite frames.

getCocktailFrames(arr)

Yields bidirectional iteration frames.

getAlgoFrames(algo, arr)

Dispatcher to fetch frames based on active selection.

Deep Dive Tab

generateDD[Algo]Frames(arr)

Specialized generators that yield micro-steps with plain-English insights and variable pointers.

renderDDCode(algo) / renderDDFrame()

Renders the deep dive slots, pointers, and highlighted code lines.

ddStepForward() / ddStepBack() / ddTogglePlay() / ddLoop()

Controls the playback engine for the Deep Dive workspace.

Pathfinding Tab

updatePfAlgoInfo() / initPfGrid() / renderPfGrid()

Bootstraps the interactive grid and loads algorithm metadata.

pfCellMouseDown(e) / pfCellMouseEnter(e)

Handles drawing walls and dragging start/end nodes.

setPfMode(mode)

Toggles between Wall, Start, and End drawing modes.

clearPfGrid() / clearPfPath() / clearPfStats()

Resets the grid state safely.

pfAnimateVisit(r, c, type)

Pushes DOM updates to grid cells based on current search frontier.

generateMaze()

Runs a randomized depth-first search to carve a perfect maze into the grid.

Custom DSL & Library

dslInitIfEmpty() / dslUpdateLineCount() / dslLoadPreset()

Sets up the user code editor.

dslSetArray() / dslLog(msg, cls)

Handles DSL inputs and console outputs.

dslPlayFromStart() / dslUpdateProgress() / dslTogglePlay()

Executes the compiled DSL frames visually.

libInitIfEmpty() / libSelect(key) / libRenderActive()

Populates the code snippet reference library.

libHighlight(code)

Applies regex-based syntax highlighting to snippets.

libCopyActive() / libRunSort(fnName, arr) / libRunActive()

Allows users to copy code or execute library snippets live in the browser.